Laboratorio 8 — Hoja de trabajo

MySQL 8.4 con PDO (CRUD con consultas preparadas)

Nombre: Ronald Salvatierra Flores **Fecha:** 14 / 10 / 2026

Comandos: `docker compose up -d --build --wait` · `composer dump-autoload` · `docker compose exec -T php php verificar.php` (objetivo: 15 de 15). El sitio queda en http://127.0.0.1:8000/productos. Las rutas de archivo de los mensajes son las del contenedor (/app).

Parte A — La base de datos

1. Entra a la consola y ejecuta SHOW CREATE TABLE productos\G. ¿Qué restricciones (CONSTRAINT) tiene la tabla?

```
docker compose exec mysql mysql -u apicultores -pmiel2026 apicultores
```

La tabla tiene cinco reglas que MySQL hace cumplir por sí mismo:

```
PRIMARY KEY (id)                                        -- cada fila tiene un id único
UNIQUE KEY uq_productos_codigo (codigo)                -- no puede haber dos códigos iguales
CONSTRAINT ck_productos_precio CHECK (precio > 0)       -- precio siempre positivo
CONSTRAINT ck_productos_stock  CHECK (stock >= 0)       -- stock nunca negativo
CONSTRAINT fk_productos_socio  FOREIGN KEY (socio_id)   -- el socio debe existir
          REFERENCES socios (id)
```

2. ¿Por qué precio es DECIMAL(10,2) y no FLOAT? Prueba en PHP: var_dump(0.1 + 0.2 == 0.3);

```
bool(false)        // 0.1 + 0.2 vale 0.30000000000000004
```

FLOAT y DOUBLE guardan los números en binario y muchos decimales (0.1, 0.2) no se pueden representar exactamente, por eso 0.1 + 0.2 no es igual a 0.3. En dinero esos errores diminutos se acumulan y los totales no cuadran. DECIMAL(10,2) guarda el valor exacto en base 10 con 2 decimales (hasta 99.999.999,99): 32.50 queda como 32.50.

3. Intenta en la consola un INSERT con precio -1 y luego con socio_id = 999. Copia los dos errores. ¿Qué ganamos con que la BD también valide?

```
mysql> INSERT INTO productos (codigo, nombre, categoria, precio, stock, socio_id) VALUES ('ZZZ-
01','X','Miel',-1,1,1);
ERROR 3819 (HY000): Check constraint 'ck_productos_precio' is violated.

mysql> INSERT INTO productos (codigo, nombre, categoria, precio, stock, socio_id) VALUES ('ZZZ-
01','X','Miel',10,1,999);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
(`apicultores`.`productos`, CONSTRAINT `fk_productos_socio` FOREIGN KEY (`socio_id`) REFERENCES `socios`
(`id`))
```

La validación de PHP es la primera línea de defensa y da mensajes amables, pero no es la única puerta a los datos: otro script, un import o un error de programación podrían saltarse el formulario. Con las reglas en la BD, los datos inválidos no entran por ningún camino. Es la última línea de defensa y mantiene la integridad sin importar quién escriba.

Parte B — Conexión (TODO 6 en src/BaseDatos/Conexion.php)

1. ¿Qué significa cada parte del DSN mysql:host=...;port=...;dbname=...;charset=utf8mb4?

```
mysql:           controlador (driver) de PDO: se habla con MySQL
host=mysql       dónde está el servidor (en Docker, el nombre del servicio del compose)
port=3306        puerto en el que escucha MySQL
dbname=apicultores   base de datos que se usa al conectar
charset=utf8mb4 codificación de la conexión: tildes, ñ y emojis se guardan y leen bien
```

2. ¿Qué hace cada opción de PDO? Completa el TODO 6.

```
PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,        // los errores de SQL son excepciones
PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,   // cada fila es un arreglo asociativo
PDO::ATTR_EMULATE_PREPARES => false,                 // TODO 6: consultas preparadas reales
```

ERRMODE_EXCEPTION: ante un error SQL PDO lanza PDOException en vez de devolver false en silencio; así los errores no pasan desapercibidos y se pueden capturar (try/catch). DEFAULT_FETCH_MODE = FETCH_ASSOC: cada fila llega como ['codigo' => 'MIE-500', …] y no duplicada con índices numéricos. EMULATE_PREPARES = false: PHP envía a MySQL primero la consulta con los marcadores y después los valores, por separado (preparación nativa); con la emulación PHP armaría el texto SQL por su cuenta. Con la preparación real la estructura de la consulta queda fijada antes de conocer los datos.

3. ¿Por qué la clave de la BD se lee de una variable de entorno (config/bd.php)?

Porque una clave escrita en el código termina en Git, en capturas y en copias del proyecto. Con getenv() el código no contiene secretos: cada entorno (mi laptop, el servidor, el contenedor) define sus propios valores (BD_HOST, BD_USUARIO, BD_CLAVE…) en docker-compose.yml o en el servidor, y se pueden cambiar sin tocar el código. Los valores por defecto del archivo sirven solo para pruebas locales.

Parte C — Inyección SQL (TODO 1 y 2 en ProductoConsultas.php)

Antes de los TODO, el texto del usuario se pegaba dentro del SQL: "WHERE p.nombre LIKE '%$texto%' OR p.codigo LIKE '%$texto%'". Resultados de las búsquedas:

```
Búsqueda               Antes de los TODO   Después de los TODO
miel                    3 productos         3 productos
zzz                     0 productos         0 productos
zzz' OR 1=1 -- -        6 productos (¡todos!) 0 productos
'  (una comilla)        0 productos         0 productos
%                       6 productos (todos) 0 productos
```

1–2. miel → ¿cuántos? zzz → ¿cuántos? Ahora zzz' OR 1=1 -- - → ¿cuántos? Escribe el SQL que se ejecutó realmente.

miel devuelve 3 (Miel multifloral 500 g, 1 kg y de cusi) y zzz devuelve 0. Pero zzz' OR 1=1 -- - devuelve los 6 productos: el atacante cerró la cadena con la comilla, añadió una condición siempre verdadera y comentó el resto con "-- -". El SQL real fue:

```
SELECT p.codigo, p.nombre, ... FROM productos p JOIN socios s ON s.id = p.socio_id
WHERE p.nombre LIKE '%zzz' OR 1=1 -- -%' OR p.codigo LIKE '%zzz' OR 1=1 -- -%'
ORDER BY p.categoria, p.nombre
```

Es una inyección SQL: lo que escribió el usuario dejó de ser un dato y pasó a ser parte de la instrucción. Con otras cargas (UNION SELECT …) se podrían leer otras tablas, y con ; DROP TABLE… borrarlas.

3. ¿Qué pasa con una comilla (') suelta? Mira datos/errores.log.

En esta consulta la comilla aparece dos veces (una por cada LIKE), así que las comillas quedan en número par y MySQL logra "emparejarlas" de otra forma: no hay error ni registro en errores.log, pero la consulta ya no es la que escribió el programador y la búsqueda devuelve 0 resultados sin explicación. Si el texto tuviera otra cantidad de comillas, sí habría un error de sintaxis y un 500. En ambos casos queda demostrado que el texto del usuario altera la estructura del SQL.

4. % → ¿cuántos? ¿Por qué?

Devuelve los 6 productos. En LIKE, % es un comodín que significa "cualquier texto", así que '%%%' coincide con todo. El usuario quería buscar un símbolo % literal y no lo logra.

5. Completa los TODO 1 y 2 y repite las 4 búsquedas. ¿Por qué con prepare() el texto ya no puede cambiar la consulta?

```
public function buscar(string $texto = ''): array
{
    $sql = self::SELECCION . ' WHERE p.nombre LIKE :patron OR p.codigo LIKE :patron2 ORDER BY
p.categoria, p.nombre';
    $patron = '%' . self::escaparComodines($texto) . '%';
    $consulta = $this->bd->prepare($sql);
    $consulta->execute(['patron' => $patron, 'patron2' => $patron]);
    return $consulta->fetchAll();
}

public static function escaparComodines(string $texto): string
{
    return addcslashes($texto, '%_\\');   // % → \%   _ → \_   \ → \\
}
```

Con prepare() la consulta se envía a MySQL una vez, con los marcadores :patron y :patron2, y MySQL la analiza y fija su estructura. Después execute() envía los valores aparte, como datos. Aunque el texto contenga comillas, OR o --, para MySQL sigue siendo un simple texto que se compara con LIKE; nunca vuelve a analizarse como SQL. Por eso zzz' OR 1=1 -- - ahora devuelve 0 productos, la comilla suelta se busca como carácter y % ya no devuelve todo (escaparComodines lo convierte en \%).

6. ¿Por qué hacen falta dos marcadores (:patron y :patron2) con el mismo valor?

Porque con EMULATE_PREPARES = false (preparación nativa de MySQL) un mismo marcador con nombre no puede repetirse en la consulta: cada aparición necesita su propio nombre. Se usan dos (uno para el nombre y otro para el código) y se les pasa el mismo valor.

Parte D — CRUD (TODO 3, 4 y 5)

1. Registra POL-100. En la consola: SELECT * FROM productos WHERE codigo='POL-100'; ¿Cómo se guardó el precio 32,5?

```
*************************** 1. row ***************************
       id: 7
   codigo: POL-100
   nombre: Polen seco 100 g
categoria: Derivados
   precio: 32.50
    stock: 15
 socio_id: 3
```

Se guardó como DECIMAL exacto: 32.50. El formulario recibe "32,5" (coma decimal), el controlador lo normaliza a 32.50 y el código se pasa a mayúsculas (pol-100 → POL-100). Tras guardar, la app responde 303 con Location: /productos y un aviso (patrón Post/Redirect/Get: si el usuario recarga la página no se reenvía el formulario).

2. TODO 3: ¿por qué traducir el error 1062 si el validador ya revisa que el código no exista?

```
} catch (PDOException $e) {
    if (($e->errorInfo[1] ?? null) === 1062) {
        throw new Conflicto("Ya existe un producto con el código {$d['codigo']}.", 0, $e);
    }
    throw $e;
}
```

Porque entre la validación (SELECT) y el INSERT hay un instante, y dos personas a la vez pueden pasar ambas la validación con el mismo código; la segunda en insertar recibe el error 1062 (entrada

duplicada) del índice UNIQUE. La base es la única que garantiza la unicidad. Se traduce a Conflicto (409/422, mensaje claro) y se conserva el PDOException como $previous, en vez de mostrar un 500.

3. TODO 4: escribe el UPDATE. ¿Qué pasaría si olvidas el WHERE?

```
UPDATE productos
   SET nombre = :nombre, categoria = :categoria, precio = :precio,
       stock = :stock, socio_id = :socio_id
WHERE codigo = :codigo
```

Sin el WHERE, el UPDATE se aplicaría a TODAS las filas: los 6 productos quedarían con el mismo nombre, precio y stock. Es uno de los errores más graves en SQL; por eso el verificador comprueba que actualizar() cambie solo el producto indicado.

4. TODO 5: ¿qué devuelve rowCount() al borrar un código que no existe? ¿Qué debe responder la aplicación?

```
$consulta = $this->bd->prepare('DELETE FROM productos WHERE codigo = ?');
$consulta->execute([$codigo]);
if ($consulta->rowCount() === 0) {
    throw new NoEncontrado("No existe el producto $codigo.");
}
```

rowCount() devuelve 0 porque no se borró ninguna fila (DELETE no da error aunque nada coincida). La aplicación debe responder 404 Not Found: "No existe el producto NO-HAY.". Repetir el borrado de un producto ya eliminado también da 404.

Parte E — Verificación

php verificar.php → objetivo 15 de 15.

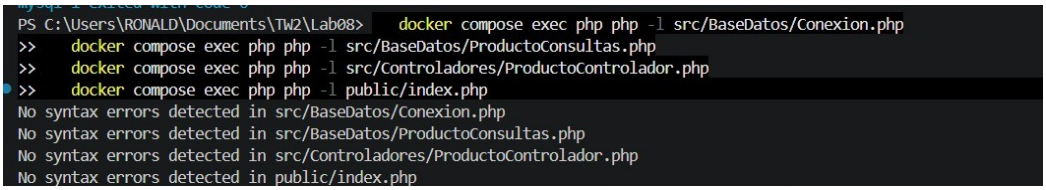


Figura 1. php -l sin errores en los archivos modificados.

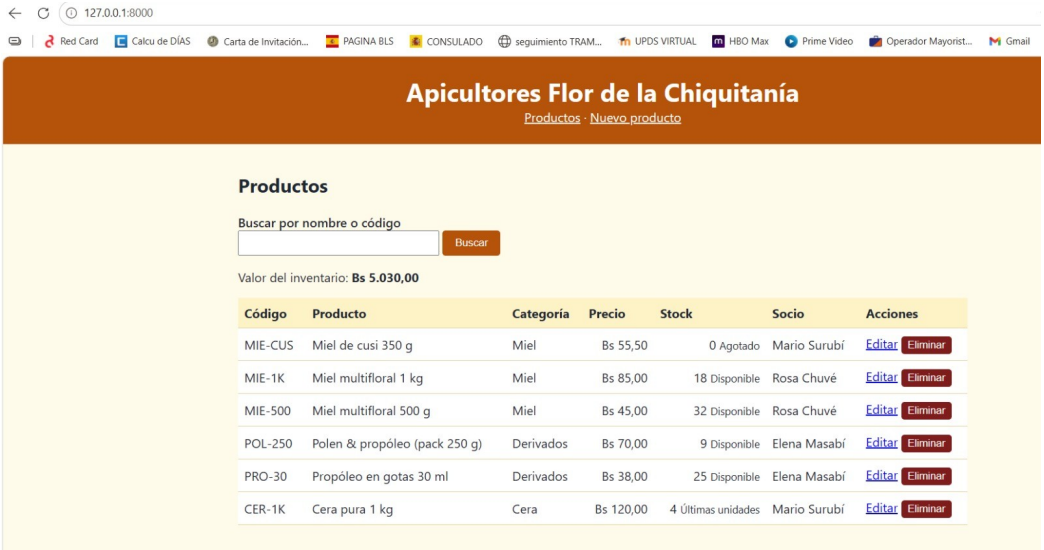


Figura 3. Catálogo en http://127.0.0.1:8000/productos con los 6 productos y el valor del inventario.

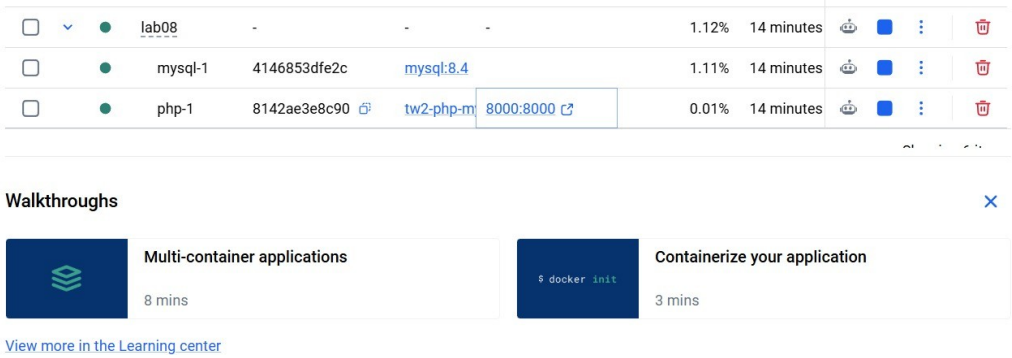


Figura 4. Docker Desktop: proyecto lab08 con los contenedores mysql-1 (mysql:8.4) y php-1 (puerto 8000).

Detén MySQL (docker compose stop) y abre la página: ¿qué ve el usuario? ¿Aparece el usuario o la clave de la BD?

```
HTTP/1.1 500 Internal Server Error
Algo salió mal
Tuvimos un problema inesperado. Si persiste, avisa a soporte con la referencia 3115c70a.

datos/errores.log:
[2026-10-15T00:55:37+00:00] ref=3115c70a PDOException: SQLSTATE[HY000] [2002] Connection refused en
/app/src/BaseDatos/Conexion.php:22
```

El usuario ve una página 500 amable con una referencia; no aparece el usuario (apicultores), la clave (miel2026), el host ni el puerto de la base. Los detalles técnicos quedan solo en datos/errores.log, junto a esa referencia. Al volver a iniciarlo con docker compose start, la página funciona de nuevo y los datos siguen ahí (están en el volumen mysql_lab08).

Reflexión

Explica con tus palabras, sin código, por qué una consulta preparada evita la inyección SQL.

Es como entregarle al cocinero una receta con espacios en blanco ya impresa ("agregar ___ gramos de sal") y darle aparte los datos para rellenarlos. Aunque alguien escriba en el espacio "sal y además incendia la cocina", el cocinero solo lo lee como la cantidad que va en ese hueco, no como una orden nueva. La base primero recibe y entiende la estructura de la consulta, y después recibe los datos por otro camino; lo que el usuario escribe jamás puede convertirse en instrucciones. Con consultas armadas pegando texto, en cambio, el cocinero lee todo como una sola receta y obedece lo que el intruso haya agregado.

Anexo — Archivos completados

Archivos de la carpeta inicio con los TODO resueltos.

docker-compose.yml

```
# Laboratorio 8: PHP + MySQL 8.4. Abre http://127.0.0.1:8000/productos
# Las credenciales son SOLO para esta base educativa local.
name: tw2-lab08

services:
  mysql:
    image: mysql:8.4
    command: --character-set-server=utf8mb4
    environment:
      MYSQL_ROOT_PASSWORD: raiz2026
      MYSQL_DATABASE: apicultores
      MYSQL_USER: apicultores
      MYSQL_PASSWORD: miel2026
    volumes:
      - mysql_lab08:/var/lib/mysql
      # Se ejecuta SOLO cuando MySQL crea un volumen vacío (primera vez)
      - ./sql:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 -u root -p$$MYSQL_ROOT_PASSWORD --silent"]
      interval: 5s
      timeout: 5s
      retries: 30
      start_period: 20s

  php:
    build: .
    image: tw2-php-mysql:8.4
    working_dir: /app
    depends_on:
      mysql:
        condition: service_healthy
    environment:
      # Dentro del contenedor, "localhost" sería el propio PHP: la base se llama "mysql".
      BD_HOST: mysql
      BD_PUERTO: "3306"
      BD_NOMBRE: apicultores
      BD_USUARIO: apicultores
      BD_CLAVE: miel2026
    volumes:
      - .:/app
    ports:
      - "127.0.0.1:8000:8000"
    command: sh -c "composer dump-autoload && php -S 0.0.0.0:8000 -t public public/index.php"

volumes:
  mysql_lab08:
```

sql/01_estructura.sql

```
-- sql/01_estructura.sql — Estructura de la base. MySQL la ejecuta sola al crear un volumen vacío.
SET NAMES utf8mb4;
```

```
CREATE TABLE IF NOT EXISTS socios (
    id     INT UNSIGNED NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(80)  NOT NULL,
    PRIMARY KEY (id),
    UNIQUE KEY uq_socios_nombre (nombre)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS productos (
    id        INT UNSIGNED  NOT NULL AUTO_INCREMENT,
    codigo    VARCHAR(10)   NOT NULL,
    nombre    VARCHAR(120)  NOT NULL,
    categoria ENUM('Miel', 'Derivados', 'Cera') NOT NULL,
    precio    DECIMAL(10,2) NOT NULL,          -- dinero: DECIMAL, nunca FLOAT
    stock     INT           NOT NULL DEFAULT 0,
    socio_id  INT UNSIGNED  NOT NULL,
    PRIMARY KEY (id),
    UNIQUE KEY uq_productos_codigo (codigo),   -- impide códigos repetidos
    CONSTRAINT ck_productos_precio CHECK (precio > 0),
    CONSTRAINT ck_productos_stock  CHECK (stock >= 0),
    CONSTRAINT fk_productos_socio  FOREIGN KEY (socio_id) REFERENCES socios (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

sql/02_datos.sql

```
-- sql/02_datos.sql — Datos iniciales: 3 socios y los 6 productos del catálogo (los mismos del lab 7).
-- Una sentencia por ";" (verificar.php vuelve a cargar este archivo para restaurar los datos de
prueba).
SET NAMES utf8mb4;

INSERT INTO socios (id, nombre) VALUES
    (1, 'Rosa Chuvé'),
    (2, 'Mario Surubí'),
    (3, 'Elena Masabí');

INSERT INTO productos (codigo, nombre, categoria, precio, stock, socio_id) VALUES
    ('MIE-500', 'Miel multifloral 500 g',        'Miel',      45.00, 32, 1),
    ('MIE-1K',  'Miel multifloral 1 kg',         'Miel',      85.00, 18, 1),
    ('MIE-CUS', 'Miel de cusi 350 g',            'Miel',      55.50,  0, 2),
    ('POL-250', 'Polen & propóleo (pack 250 g)', 'Derivados', 70.00,  9, 3),
    ('PRO-30',  'Propóleo en gotas 30 ml',       'Derivados', 38.00, 25, 3),
    ('CER-1K',  'Cera pura 1 kg',                'Cera',     120.00,  4, 2);
```

config/bd.php

```
<?php
// config/bd.php — Datos de conexión. Se leen de variables de ENTORNO: nunca se escriben claves en el
código.
// Dentro de Docker las define docker-compose.yml (BD_HOST=mysql). Los valores por defecto sirven solo
para pruebas locales.
declare(strict_types=1);

return [
    'host'    => getenv('BD_HOST') ?: '127.0.0.1',
    'puerto'  => (int) (getenv('BD_PUERTO') ?: 3306),
    'nombre'  => getenv('BD_NOMBRE') ?: 'apicultores',
    'usuario' => getenv('BD_USUARIO') ?: 'apicultores',
```

```
    'clave'   => getenv('BD_CLAVE') ?: 'miel2026',
];
```

src/BaseDatos/Conexion.php

```
<?php
// src/BaseDatos/Conexion.php — Crea UNA conexión PDO a MySQL con las opciones seguras del curso.
declare(strict_types=1);

namespace Apicultores\BaseDatos;

use PDO;

final class Conexion
{
    private static ?PDO $pdo = null;

    /** @param array{host:string,puerto:int,nombre:string,usuario:string,clave:string}|null $config */
    public static function obtener(?array $config = null): PDO
    {
        if (self::$pdo !== null && $config === null) {
            return self::$pdo;
        }
        $config ??= require __DIR__ . '/../../config/bd.php';
        $dsn = sprintf('mysql:host=%s;port=%d;dbname=%s;charset=utf8mb4', $config['host'],
$config['puerto'], $config['nombre']);

        return self::$pdo = new PDO($dsn, $config['usuario'], $config['clave'], [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,        // los errores de SQL son excepciones
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,   // cada fila es un arreglo asociativo
            // TODO 6: consultas preparadas REALES del servidor: la estructura SQL y los datos viajan
por separado.
            PDO::ATTR_EMULATE_PREPARES => false,
        ]);
    }
}
```

src/BaseDatos/ProductoConsultas.php

```
<?php
// src/BaseDatos/ProductoConsultas.php — TODO el SQL de la tabla productos vive aquí. Solo consultas
PREPARADAS.
// Regla de oro: los DATOS van en marcadores (:nombre o ?); nunca se concatenan en el texto SQL.
declare(strict_types=1);

namespace Apicultores\BaseDatos;

use Apicultores\Excepciones\Conflicto;
use Apicultores\Excepciones\NoEncontrado;
use PDO;
use PDOException;

final class ProductoConsultas
{
    private const SELECCION = 'SELECT p.codigo, p.nombre, p.categoria, p.precio, p.stock, p.socio_id,
s.nombre AS socio
                               FROM productos p JOIN socios s ON s.id = p.socio_id';
```

```
    public function __construct(private readonly PDO $bd)
    {
    }

    /** @return array<int, array<string, mixed>> */
    public function todos(): array
    {
        return $this->bd->query(self::SELECCION . ' ORDER BY p.categoria, p.nombre')->fetchAll();
    }

    /** @return array<int, array<string, mixed>> */
    public function socios(): array
    {
        return $this->bd->query('SELECT id, nombre FROM socios ORDER BY nombre')->fetchAll();
    }

    public function contar(): int
    {
        return (int) $this->bd->query('SELECT COUNT(*) FROM productos')->fetchColumn();
    }

    public function valorInventario(): float
    {
        return (float) $this->bd->query('SELECT COALESCE(SUM(precio * stock), 0) FROM productos')-
>fetchColumn();
    }

    public function porCodigo(string $codigo): ?array
    {
        $consulta = $this->bd->prepare(self::SELECCION . ' WHERE p.codigo = :codigo');
        $consulta->execute(['codigo' => $codigo]);
        return $consulta->fetch() ?: null;
    }

    public function existeSocio(int $id): bool
    {
        $consulta = $this->bd->prepare('SELECT 1 FROM socios WHERE id = ?');
        $consulta->execute([$id]);
        return $consulta->fetchColumn() !== false;
    }

    // TODO 1: búsqueda con LIKE y marcadores (:patron y :patron2: con preparación nativa no se repite
un mismo nombre).
    public function buscar(string $texto = ''): array
    {
        $sql = self::SELECCION . ' WHERE p.nombre LIKE :patron OR p.codigo LIKE :patron2 ORDER BY
p.categoria, p.nombre';
        $patron = '%' . self::escaparComodines($texto) . '%';
        $consulta = $this->bd->prepare($sql);
        $consulta->execute(['patron' => $patron, 'patron2' => $patron]);
        return $consulta->fetchAll();
    }

    // TODO 2: % y _ son comodines de LIKE; se escapan para buscarlos como texto literal (no es el
escape HTML).
```

```
    public static function escaparComodines(string $texto): string
    {
        return addcslashes($texto, '%_\\');
    }

    // TODO 3: la base es la ÚLTIMA línea de defensa contra códigos repetidos.
    public function insertar(array $d): int
    {
        try {
            $consulta = $this->bd->prepare(
                'INSERT INTO productos (codigo, nombre, categoria, precio, stock, socio_id)
                 VALUES (:codigo, :nombre, :categoria, :precio, :stock, :socio_id)');
            $consulta->execute([
                'codigo' => $d['codigo'], 'nombre' => $d['nombre'], 'categoria' => $d['categoria'],
                'precio' => $d['precio'], 'stock' => $d['stock'], 'socio_id' => $d['socio_id'],
            ]);
        } catch (PDOException $e) {
            // 1062 = entrada duplicada (UNIQUE): aunque la validación revisó el código,
            // otra persona pudo registrarlo un instante antes.
            if (($e->errorInfo[1] ?? null) === 1062) {
                throw new Conflicto("Ya existe un producto con el código {$d['codigo']}.", 0, $e);
            }
            throw $e;
        }
        return (int) $this->bd->lastInsertId();
    }

    // TODO 4: WHERE limita el cambio a UN producto. Sin WHERE se actualizarían todos.
    public function actualizar(string $codigo, array $d): void
    {
        $consulta = $this->bd->prepare(
            'UPDATE productos SET nombre = :nombre, categoria = :categoria, precio = :precio, stock
= :stock, socio_id = :socio_id
             WHERE codigo = :codigo');
        $consulta->execute([
            'nombre' => $d['nombre'], 'categoria' => $d['categoria'], 'precio' => $d['precio'],
            'stock' => $d['stock'], 'socio_id' => $d['socio_id'], 'codigo' => $codigo,
        ]);
    }

    // TODO 5: rowCount() dice cuántas filas afectó: 0 significa que no existía ese código.
    public function eliminar(string $codigo): void
    {
        $consulta = $this->bd->prepare('DELETE FROM productos WHERE codigo = ?');
        $consulta->execute([$codigo]);
        if ($consulta->rowCount() === 0) {
            throw new NoEncontrado("No existe el producto $codigo.");
        }
    }
}
```